SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

A PROJECT REPORT

on

AI Based Web Application Vulnerability Detection and Risk Assessment

Unique ID : E0224033

Student Name : G L Swaminathan

Year/Sem: 3rd year / 5th sem

Stream : B. Tech CSE Cyber Security and Internet of Things

Course Code : CYB23DLU02

Course Name : INFORMATION SECURITY LABORATORY

Academic Year : 2026-2027

Course Faculty : Dr. S. Surya

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

A PROJECT REPORT

on

AI Based Web Application Vulnerability Detection and Risk Assessment

Unique ID : E0224033

Student Name : G L Swaminathan

Year/Sem: 3rd year / 5th sem

Stream : B. Tech CSE Cyber Security and Internet of Things

Course Code : CYB23DLU02

Course Name : INFORMATION SECURITY LABORATORY

Academic Year : 2026-2027

Course Faculty : Dr. S. Surya

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report entitled "AI Based Web Application Vulnerability Detection and Risk Assessment" is the bonafide record of work done by **"G L Swaminathan - E0224033",** who carried out the work under my supervision as part of the course "Information Security" in partial fulfillment of the requirements for the award of the degree in Computer Science.

Signature of the Course Faculty

Dr. S. Surya
Assistant Professor,
Department of Cyber Security and Internet of Things
Sri Ramachandra Faculty of Engineering and
Technology,
SRIHER, Porur, Chennai-600 116.

Evaluation Date:

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF TABLES	**v**
	LIST OF FIGURES	**vi**
	ABSTRACT	**vii**
1	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Motivation	2
	1.4 Objectives	2
	1.5 Scope	3
	1.6 Justification	3
2	**LITERATURE SURVEY**	**4**
3	**EXISTING SYSTEM**	**6**
4	**PROPOSED SYSTEM**	**8**
5	**SYSTEM ARCHITECTURE**	**11**
6	**MODULE DESCRIPTION**	**13**
7	**THREAT MODEL**	**19**
8	**VULNERABILITY ANALYSIS**	**21**
9	**RISK ASSESSMENT**	**22**
10	**IMPLEMENTATION**	**23**
11	**SECURITY IMPLEMENTATION**	**26**
12	**SECURITY TESTING**	**27**
13	**RESULTS AND ANALYSIS**	**28**
14	**CONCLUSION**	**31**
15	**FUTURE ENHANCEMENT**	**32**
16	**REFERENCES**	**33**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Literature Survey	4
3.1	Existing System Comparison	6
7.1	Threats Affecting Major Assets	20
8.1	Vulnerability Analysis	21
9.1	Risk Register	22
12.1	Security Test Cases	27
13.1	Results Table / Metrics	28

LIST OF FIGURES

FIG. NO.	TITLE	PAGE NO.
4.1	Proposed Workflow	9
5.1	System Architecture	11
5.2	Data Flow Diagram (Level 1)	12
10.1	User Login Interface (live VERITAS)	24
10.2	User Projects Workspace (live production VERITAS)	24
10.3	Administrator Dashboard (live VERITAS)	25
13.1	Live Production Scan Findings — NurseQuest	29
13.2	Live Production Home — Findings by Run and Severity	30
13.3	Live Production Scan Findings — AI Honeypot	30

ABSTRACT

In this project, we attempt to study how static analysis and related research methods can help detect web application vulnerabilities earlier in the development lifecycle. Web applications are built from source code, third-party libraries, configuration files, and secrets; weaknesses such as injection flaws, insecure authentication, secret leakage, and vulnerable dependencies may affect confidentiality, integrity, or availability. From our literature survey of journal and conference papers, we observed that many research systems improve detection accuracy, while fewer student-oriented platforms combine a GitHub workspace, multi-engine scanning, AI reporting, and residual-risk tracking.

We therefore attempt to design and implement AI Based Web Application Vulnerability Detection and Risk Assessment under the system name VERITAS. The platform is a single-server web application in which users connect owned GitHub repositories, run multi-engine vulnerability detection (secrets scanning, software composition analysis, and static analysis), optionally use AI-assisted review and report writing, score findings by severity and project criticality, and track status, suppressions, and remediation. The workflow covers account access and authorization, project workspace, scan execution, AI triage, findings management, and dashboard/reporting with multi-format export.

Evaluation is limited to an authorized laboratory setting. Experimental results discussed in this report use the NurseQuest test repository as a sample target, together with other permitted sample repositories. We describe the end-to-end flow from signup and admin approval through repository linking, engine scans, AI summarization, risk-sorted findings, controls, re-scan/compare, and result discussion. We do not invent or claim production-scale accuracy numbers beyond laboratory evidenc

1. INTRODUCTION

1.1 Background

In this project, we attempt to address the practical difficulty of finding and prioritizing web application vulnerabilities in Git-hosted source code. Student and small development teams commonly deliver features through web applications stored in repositories; source code, dependency manifests, and configuration files can introduce defects before deployment. Manual review does not scale well, and raw scanner output without risk context is difficult to use in a lab report. We therefore work on an AI-assisted vulnerability detection and risk assessment platform that links detection, prioritization, and remediation tracking.

VERITAS implements AI Based Web Application Vulnerability Detection and Risk Assessment using a FastAPI backend and a React single-page application served from the same origin. Users log in with session cookies, connect owned GitHub repositories, create projects, and start scans. Deterministic engines (for example Gitleaks, OSV-Scanner, and Semgrep) detect secrets, known CVEs, and SAST issues. Optional OpenRouter and Groq stages add structured review text and remediation narrative when API keys are configured. Findings are shown in a risk-sorted register and in user/admin dashboards.

1.2 Problem Statement

Web applications continuously receive new commits, dependency updates, and configuration changes. Without a controlled process, vulnerabilities may remain unnoticed. In a laboratory setting we need a practical way to detect issues early in the repository, assess severity and criticality-weighted risk, and check whether remediation reduces residual exposure.

In many existing setups, scanners, ticket notes, and informal documents remain separate. This makes it hard to keep a clear link between the repository asset, vulnerability class, severity, risk score, recommended control, and remaining open findings. If ownership checks, audit logs, and session security are weak, the assessment platform itself can become an attack surface.

In this project, AI Based Web Application Vulnerability Detection and Risk Assessment addresses these gaps by bringing together GitHub-linked projects, multi-engine vulnerability detection, optional AI-assisted triage and reporting, risk scoring, finding workflow (open/triage/fixed/false_positive), suppressions, scan compare, and monitoring dashboards inside one authenticated platform ({SYSTEM}).

1.3 Motivation

The growth of web APIs, SPA frontends, and open-source dependencies increases the number of possible vulnerabilities. Information Security coursework needs a realistic but safe environment where students can practice detection, risk assessment, and verification of countermeasures.

Running scanners only from the command line often produces unstructured results that are hard to compare across commits. Research on machine learning and large language models for vulnerability detection is active, but for a lab platform AI should remain assistive and fail-open so that deterministic engines remain the primary signal. This combination motivates the present work.

The proposed VERITAS platform is motivated by the need for a secure, centralized laboratory system that detects web-application vulnerabilities in linked repositories, assesses risk, supports remediation tracking, records residual open findings, and presents evidence through reports and dashboards.

1.4 Objectives

The primary objectives of AI Based Web Application Vulnerability Detection and Risk Assessment are:

- To provide secure account access with email verification, admin approval, and session management.
- To maintain a user workspace of projects linked to owned GitHub repositories (application assets).
- To detect secrets, vulnerable dependencies, and SAST issues using industry engines.
- To optionally enrich scans with AI code review and AI-generated remediation reports.
- To score and prioritize findings using severity and project criticality.
- To support finding workflow, suppressions, cancel, branch/diff scope, and scan quotas.
- To compare scans and measure residual risk after remediation and retesting.
- To present dashboards, notifications, and exportable reports (Markdown, JSON, CSV, HTML, SARIF).
- To enforce server-side authorization, encrypted GitHub tokens, audit logging, and webhook integrity.

- To enable auto-scan on push for continuous laboratory assessment.
- To demonstrate measurable risk reduction through controlled security testing.
- To document architecture, threat model, and results suitable for academic evaluation.

1.5 Scope

The scope of AI Based Web Application Vulnerability Detection and Risk Assessment is to design and implement a laboratory-ready web platform that detects vulnerabilities in GitHub-hosted web application repositories and assesses associated security risk.

In scope: authentication and RBAC (admin/user), project CRUD, GitHub OAuth with ownership checks, async multi-engine scans, AI-assisted review/reporting when API keys are configured, findings management, risk summaries, suppressions, compare, quotas, push webhooks, and report export/share links. Scans run against authorized sample repositories in a controlled environment.

Out of scope for this phase: full DAST/HTTP corpus exploitation, organization-wide GitHub Apps beyond user OAuth, local LLM hosting requirements, and production multi-region HA. The platform is intended for authorized academic/laboratory use with synthetic or permitted repositories only.

1.6 Justification

This topic was selected because web application vulnerabilities remain among the most common and impactful security issues, while AI can accelerate triage without replacing trusted static engines. A single platform that connects detection to residual-risk tracking matches real DevSecOps practice and Information Security laboratory learning outcomes.

The work applies core concepts—authentication, authorization, secure session handling, vulnerability classification (OWASP/CWE), severity and risk scoring, controls, residual risk, auditability, and secure integration with third-party APIs—within one coherent implementation (VERITAS).

2. LITERATURE SURVEY

Table 2.1: Literature Survey

Author and Year	Methodology / Materials and Methods	Datasets	Results / Outcome	Advantage / Key findings from the work	Disadvantage / Gap Identified from the work
Medeiros, Neves, Correia (2016)	Static analysis + data mining to detect/remove web app vulnerabilities	Web application vulnerability corpora used in the study	Improved detection with reduced false positives vs rule-only SA	Shows data-mining can refine static analysis for web flaws	Not a multi-engine lab platform with residual-risk workflow
Medeiros et al., DEKANT (ISSTA 2016)	Learning-based static analysis tool for web vulnerability detection	Annotated web application code / ISSTA evaluation setup	Learned patterns detect web application vulnerabilities	ACM conference prototype for automated learning of vuln patterns	No GitHub workspace, AI triage product layer, or residual tracking
Fang et al., TAP (2019)	PHP tokenizer + Word2Vec + LSTM classification	SARD (CWE-89), SQLI-LABS	High AUC/accuracy reported for PHP vuln classification	Token+LSTM model effective for PHP static detection	PHP-centric research model; no SCA/secrets/residual risk product
Lin et al., VulEye (2023)	PDG/SDG slicing + GNN (GCN, pooling, attention) classifier	SARD PHP vulnerability test suite; lab XSS/SQLi apps	Strong macro-F1 on binary/multiclass SARD tasks	Graph structure helps locate sink-related vulnerable slices	Detection-focused; lacks integrated risk register & controls
Cao et al., RealVul (2024)	LLM fine-tuning with candidate localization & normalization	Vulnerability data from ~180 PHP projects (paper evaluation)	Improved LLM detection effectiveness/generalization (preprint)	Demonstrates LLMs for web vuln detection research	Preprint research pipeline; not fail-open engine+AI lab ASPM

1. Medeiros, Neves, and Correia (2016) in IEEE Transactions on Reliability combine static analysis with data mining to detect and remove web application vulnerabilities. Their work shows that machine-learning-assisted static analysis can reduce false positives compared with purely hand-crafted rules. The focus is detection/removal in application code rather than a full multi-user scan workspace with residual-risk tracking after controls.
2. Medeiros et al. (2016) present DEKANT at ACM ISSTA. DEKANT learns vulnerability patterns from annotated code and applies static analysis to detect web application flaws. It is an important research prototype for learning-based static detection, but it does not provide GitHub-linked project management, multi-engine orchestration, or AI triage reports inside an authenticated lab platform.

3. Fang, Han, Huang, and Wu (2019) propose TAP in PLOS ONE (PMC6860437). TAP tokenizes PHP source, uses parameter iteration for data-flow style analysis, embeds tokens with Word2Vec, and classifies with LSTM. Reported results on SARD/CWE-89 and SQLI-LABS are strong for PHP vulnerability classification. The gap for our project is that TAP is a PHP-centric research model, not an end-to-end risk assessment product covering secrets, SCA, and residual risk.
4. Lin, Xu, Fang, and Liu (2023) introduce VulEye in Applied Sciences. VulEye builds program dependence graphs, slices them into sub-dependence graphs around sensitive sinks, and classifies with a graph neural network. It advances graph-based PHP vulnerability detection on SARD-style suites. Limitations for laboratory ASPM-style use include language focus and the absence of integrated scoring, suppressions, and before/after residual-risk comparison.
5. Cao, Liao, and Shang (2024) describe RealVul (arXiv:2410.07573), an LLM-based framework for web (PHP) vulnerability detection using candidate localization, normalization, and fine-tuned code LLMs. As a preprint/research system it shows the promise of LLMs for detection. In VERITAS we treat LLM assistance as optional triage/reporting around deterministic engines, rather than as the sole detector, and we add project criticality, finding workflow, and residual-risk views.

3. EXISTING SYSTEM

Table 3.1: Existing System Comparison

Author / Year	Methodology / Materials & Methods	Datasets	Results / Outcome	Advantage / Key Findings	Disadvantage / Gap Identified
Medeiros et al. (2016)	Static analysis refined by data mining	Web vuln study datasets	Detection + removal support with fewer FPs	Strong research baseline for SA + ML	No authenticated scan product or residual risk UI
DEKANT / Medeiros et al. (2016)	Learns vulnerability patterns for static detection	ISSTA evaluation corpora	Automated learning-based web vuln detection	Conference-validated learning static analyzer	No multi-engine orchestration or AI report workflow
TAP / Fang et al. (2019)	Tokenization + LSTM deep learning for PHP	SARD, SQLI-LABS	High reported classification metrics	Accessible deep-learning PHP detector	Language-limited; no criticality-weighted residual risk
VulEye / Lin et al. (2023)	GNN over dependence-graph slices	SARD PHP suites	High F1; sink-oriented localization	Graph ML improves structural vuln clues	Not an end-to-end lab risk assessment system
RealVul / Cao et al. (2024)	LLM-based web vulnerability detection framework	Large PHP project vulnerability samples	Better LLM detection vs prior methods (preprint)	Motivates AI assistance for web vulns	Needs product security, engines, residual-risk layer (VERITAS)

3.1 Static Analysis with Data Mining (Medeiros et al., 2016)

Medeiros, Neves, and Correia (IEEE Transactions on Reliability, 2016) show that static analysis combined with data mining can detect and help remove web application vulnerabilities with fewer false positives than rule-only tools. The existing system is strong as a research detection method, but it does not offer an authenticated multi-user workspace, GitHub ownership checks, or residual risk registers after remediation.

3.2 DEKANT Learning-Based Static Detection (ISSTA 2016)

DEKANT (ACM ISSTA 2016) learns to detect web application vulnerabilities from annotated examples and applies static analysis. This is a valuable conference contribution for automated detection. Compared with the proposed VERITAS system, DEKANT does not manage scan jobs across secrets/SCA/SAST engines, AI remediation reports, or scan-to-scan compare for residual risk.

3.3 TAP Token and LSTM Detection (Fang et al., 2019)

TAP (PLOS ONE, 2019) uses PHP tokens and an LSTM classifier and reports high accuracy on public PHP vulnerability datasets. As an existing research system it targets detection quality for PHP. It does not address laboratory needs such as RBAC, encrypted OAuth tokens, criticality-weighted risk scores, suppressions, or dashboard export (SARIF/Markdown).

3.4 VulEye Graph Neural Network Detection (2023)

VulEye (Applied Sciences, 2023) uses GNN classifiers over dependence-graph slices and reports strong F1 scores on SARD PHP suites. Graph learning improves structural understanding of sinks. However, VulEye remains a detection-focused research approach and does not implement the full identify–scan–triage–control–residual-risk workflow required in this project.

3.5 RealVul LLM-Based Web Vulnerability Detection (2024)

RealVul (arXiv:2410.07573, 2024) explores whether LLMs can detect vulnerabilities in web applications, with PHP-oriented sampling and fine-tuning. It motivates AI assistance for security analysis. Gaps relative to VERITAS include reliance on research LLM pipelines, limited product security controls around the analyzer itself, and no built-in residual-risk tracking after analyst controls.

3.6 Existing System Gap

Across these research systems, the common gap is the missing laboratory-oriented product layer: secure identity, GitHub-linked assets, multi-engine detection (not only PHP ML models), optional fail-open AI triage, severity×criticality scoring, findings workflow, and residual open-risk comparison after fixes. AI Based Web Application Vulnerability Detection and Risk Assessment (VERITAS) is proposed to fill this gap for academic evaluation.

4. PROPOSED SYSTEM

4.1 Proposed Solution

The proposed solution is VERITAS: a single-origin web application where FastAPI serves both JSON APIs and the built React SPA (light theme). Users manage projects tied to owned GitHub repos, run deep multi-engine scans with optional AI markdown reports, review risk-sorted findings, update status/suppressions, export SARIF and other formats, and optionally auto-scan on push. Admins approve users, review audit trails, browse host files safely, and monitor live HTTP attack telemetry on `/admin/security` with optional Security AI analysis.

4.2 Security Mechanisms

The proposed system includes the following security and detection mechanisms:

- Argon2 password hashing, hashed sessions/OTTs, HttpOnly cookies, and admin approval gates.
- Server-side RBAC separating /admin and /user surfaces with ownership checks on workspace APIs.
- Encrypted GitHub tokens at rest and live ownership verification before repo attach.
- Multi-engine vulnerability detection: Gitleaks (secrets), OSV (SCA), Semgrep (SAST).
- Optional AI code review (OpenRouter) and AI final report/remediations (Groq), fail-open.
- Risk scoring from severity × project criticality; policy fail_severity gates.
- Finding workflow, fingerprint suppressions, scan cancel, branch/changed-file scope, quotas.
- HMAC-verified GitHub push webhooks for continuous reassessment.
- Audit events with geolocation consensus metadata; rate limits on auth endpoints.

4.3 Proposed Workflow

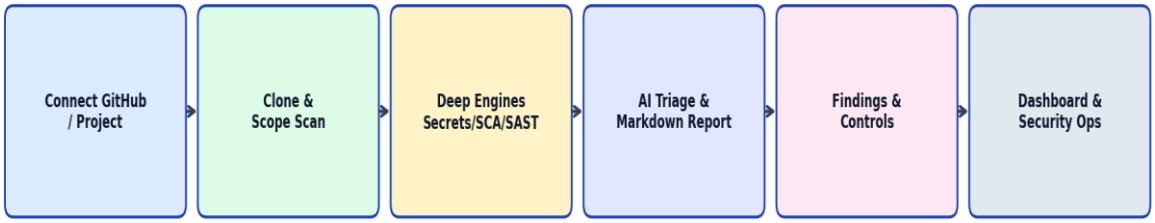


Fig 4.1: Proposed Workflow

4.4 Technologies

VERITAS is implemented as a web application: Vite + React frontend built into `backend/app/web/static/spa`, FastAPI application packages under `backend/app`, and SQLAlchemy models persisted to SQLite (lab) or PostgreSQL (production-ready). Resend delivers transactional email; GitHub OAuth and optional AI providers integrate via server-side secrets.

The main technologies used in the project are:

- Frontend: React (Vite), TypeScript/JS SPA for auth, admin, and user workspace.
- Backend: Python FastAPI, SQLAlchemy, Argon2, scan runner services and engine wrappers.
- Database: SQLite for laboratory; PostgreSQL via DATABASE_URL for production configuration.
- Integrations: GitHub OAuth/API/webhooks; Resend email; OpenRouter & Groq AI; deep scanners (Gitleaks, OSV, Semgrep, Trivy, Checkov, Bandit, …).
- Tooling: Git, npm, uvicorn, pytest, VS Code/Cursor, Chrome/Edge browsers.

4.5 Expected Security Improvements

The proposed platform is expected to improve web application security assessment in the laboratory by providing structured detection and risk tracking. Expected improvements include:

- Earlier discovery of secrets, CVEs, and SAST issues in GitHub-linked web apps.
- Consistent severity classification and criticality-weighted risk scores.
- AI-assisted remediation guidance without blocking deterministic results.
- Clear residual-risk view via open findings after fixes and suppressions.
- Traceable audit of auth, admin, and workspace security actions.
- Reduced manual collation of scanner CLI outputs.
- Comparable scans across commits for evidence of risk reduction.
- Continuous reassessment through optional auto-scan on push.
- Hardened platform security suitable for multi-user laboratory deployment.

5. SYSTEM ARCHITECTURE

5.1 Architecture Overview

The system follows a single-server, three-layer web architecture with a light-theme React SPA (public auth, `/admin/*` including Security and Files, `/user/*`). The application layer is FastAPI routing to auth, admin, workspace, webhook, HTTP telemetry, and report services. The data layer stores users, sessions, projects, scans, findings, suppressions, notifications, encrypted GitHub connections, and metadata-only `http_request_events`. SPA HTML routes never collide with JSON API paths.

5.2 Architecture Diagram

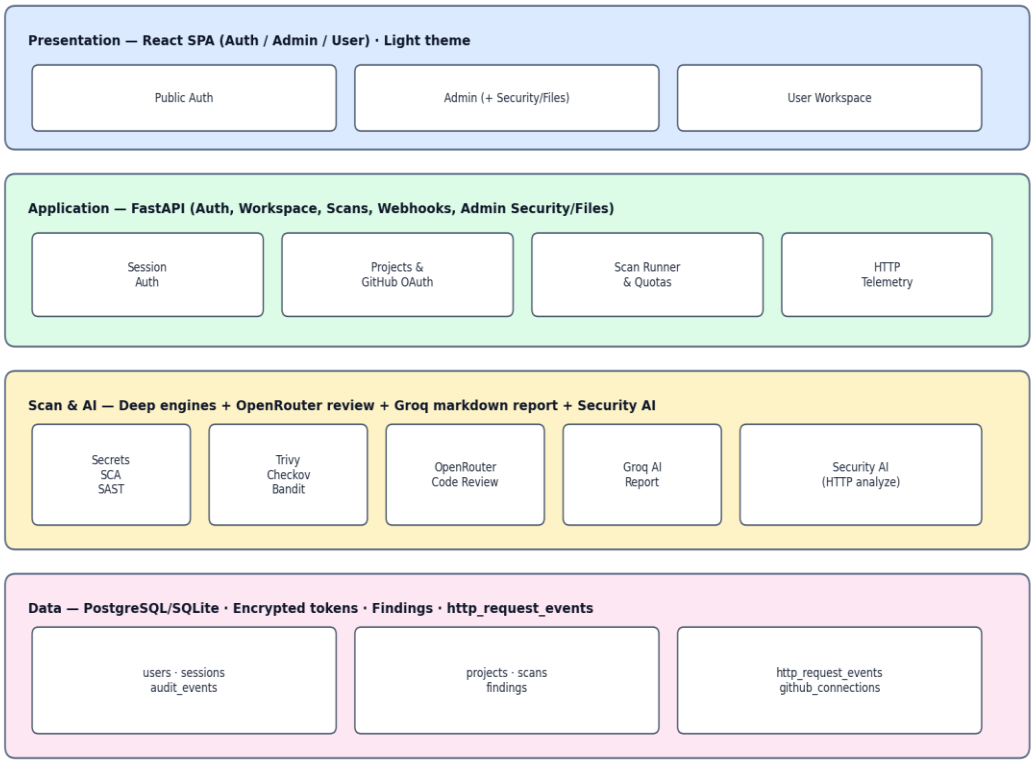


Fig 5.1: System Architecture

5.3 Data Flow

Figure 5.2 (below) is the Level-1 data flow: Users and Admins interact with Auth, Workspace, and Security Telemetry; scan jobs invoke deep engines and optional AI services; GitHub emits push webhooks; and data stores hold users, projects/scans, findings, and HTTP events used by reporting and the admin Security dashboard.

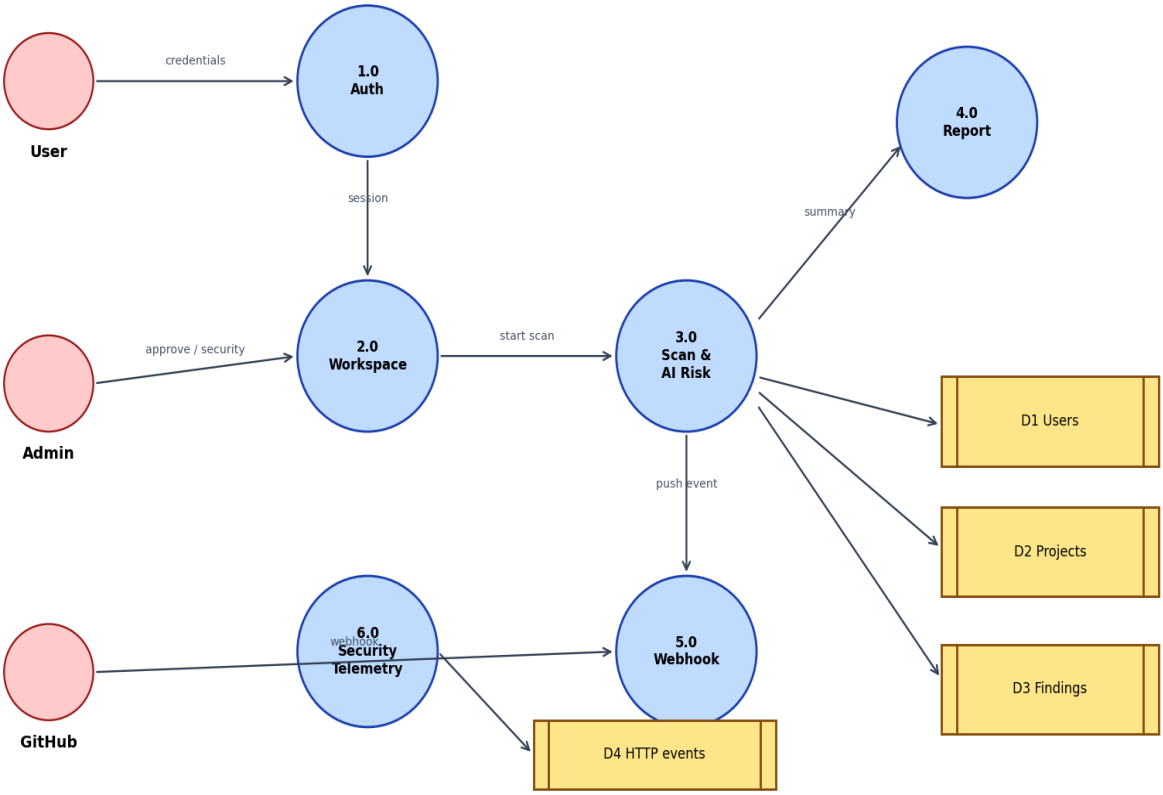


Fig 5.2: Data Flow Diagram (Level 1)

6. MODULE DESCRIPTION

AI Based Web Application Vulnerability Detection and Risk Assessment (VERITAS) is organized into eight modules aligned with identity, asset (project) management, detection, AI risk assessment, controls/residual risk, register, and monitoring.

6.1 Authentication and User Management Module

Purpose: Provides secure signup, email verification, admin approval, login sessions, password reset, and profile management.

Inputs:

- User credentials and profile fields
- Email verification / reset tokens
- Admin approve/reject/deactivate actions
- Session cookie issuance parameters

Processing:

- Validates input; hashes passwords (Argon2); transitions account status.
- Creates hashed sessions; revokes sessions on password reset.
- Writes audit events for security-relevant auth/admin actions.

Security Mechanism:

- HttpOnly / SameSite cookies; Secure in production.
- Rate limiting on auth endpoints; generic messages where enumeration matters.

Output:

- Active authenticated users and an auditable admin directory.

6.2 Project Workspace (Web Application Asset) Module

Purpose: Maintains projects that represent web application repositories under assessment.

Inputs:

- Project metadata and criticality
- GitHub OAuth authorization code
- Selected owned repository identifiers

Processing:

- Persists projects owned by the authenticated user.
- Encrypts GitHub tokens; lists only owned repos after live API checks.

Security Mechanism:

- Rejects forged github_repo_id values; tokens never sent to the client.

Output:

- Authorized project inventory ready for scanning.

6.3 Threat and Scan-Scope Identification Module

Purpose: Defines the attack surface and scan plan for a repository (branch, full vs changed files, engines).

Inputs:

- Project and git references
- Scan mode and engine subset
- Path excludes and security_level

Processing:

- Resolves refs; computes changed paths when scoped.
- Maps selected engines to threat classes (secrets, SCA, SAST).

Security Mechanism:

- Enforces scan rate/concurrency quotas (HTTP 429).

- Re-validates project ownership before job creation.

Output:

- A concrete, authorized scan job definition.

6.4 Vulnerability Detection Engine Module

Purpose: Executes deterministic scanners against a short-lived clone of the linked repository.

Inputs:

- Repository clone / worktree
- Engine configuration and excludes
- Optional cancel flag between phases

Processing:

- Runs deep engine stack (soft-skip if missing): secrets (Gitleaks/TruffleHog/detect-secrets), SCA (OSV/pip-audit/Trivy), IaC/config (Checkov/Hadolint/ShellCheck), SAST (Semgrep/Bandit/njsscan).
- Filters scanner noise (.git/tests/keyword FPs) and normalizes findings to a common schema.

Security Mechanism:

- Deletes temporary workdirs under VERITAS_SCAN_ROOT after completion.

Output:

- Normalized vulnerability findings with severity and locations.

6.5 AI Risk Assessment and Reporting Module

Purpose: Applies optional AI review and produces structured markdown risk reports and remediations.

Inputs:

- Engine findings and hot files
- AI provider configuration
- Project criticality for scoring
- scan_mode / code_review flags
- Prior open suppressions

Processing:

- Calls OpenRouter for structured review findings when enabled.
- Calls Groq for markdown AI final report (rendered in UI) and countermeasures; computes risk_score.

Security Mechanism:

- API keys remain server-side; AI failures do not drop engine findings.
- Applies suppression fingerprints automatically.

Output:

- Scored findings plus summary.ai_report markdown and by_severity metrics.

6.6 Security Controls and Residual Risk Module

Purpose: Captures analyst controls (fix, suppress, fix) and reflects residual open risk.

Inputs:

- Finding identifiers and new status
- Suppression requests
- fail_severity policy
- Compare scan pair identifiers

Processing:

- Updates finding lifecycle; stores suppressions for future scans.
- Compares fingerprints across scans; evaluates severity policy gates.

Security Mechanism:

- Owner-only authorization on finding mutations.
- Validates status enums and policy values server-side.

Output:

- Residual open findings and compare deltas after controls.

6.7 Findings Register and Export Module

Purpose: Serves as the structured vulnerability/risk register for each scan.

Inputs:

- Persisted findings and scan summary JSON
- Filter criteria (severity/status/engine)
- Export format selection
- Optional public share token
- GitHub blob link metadata
- AI remediation blocks
- Coverage / engine meta
- User identity for authz

Processing:

- Aggregates register rows; generates export payloads (incl. SARIF 2.1.0).
- Serves shareable report pages without session when token valid.

Security Mechanism:

- Capability-based report links; no raw secrets exported.
- Consistent ordering by risk_score.

Output:

- Complete register views and portable evidence packages.

6.8 Dashboard and Continuous Monitoring Module

Purpose: Visualizes security posture (user scan charts + admin Security/HTTP telemetry) and supports continuous reassessment.

Inputs:

- Aggregated user/admin metrics and ApexCharts run comparisons
- Scan history, notifications, and markdown AI reports
- Webhook push payloads and HTTP request classifications
- Audit events; admin Files browse; Security AI analyze

Processing:

- Renders light-theme dashboards, severity line charts, and Render-style scan history.
- Registers/verifies GitHub webhooks; collects metadata-only HTTP telemetry for /admin/security.

Security Mechanism:

- HMAC webhook verification; path-safe admin Files mounts; no request bodies stored in telemetry.
- Security AI (Groq/OpenRouter) suggests countermeasures; fail-open without keys.

Output:

- Operational dashboards, live security graphs, and continuous scan signals.

7. THREAT MODEL

7.1 Threat Model Overview

The threat model for AI Based Web Application Vulnerability Detection and Risk Assessment considers both the target web applications under scan and the VERITAS platform itself. Assets, threat sources, and abuse paths are analyzed so that product security controls align with the vulnerabilities the system is designed to find.

7.2 Important Assets

The major assets considered in the proposed system include:

- GitHub-linked web application source repositories
- Dependency manifests and lockfiles
- Secrets and credentials in code/history
- User accounts and session tokens on VERITAS
- Encrypted GitHub OAuth tokens
- Findings, reports, and share tokens
- Audit logs and admin approval workflow
- Webhook secrets and public app URL endpoints

7.3 Possible Threat Sources

The major threat sources considered are:

- External attackers targeting web apps or the platform
- Malicious or curious laboratory users
- Account takeover via credential stuffing
- Supply-chain / dependency abuse in scanned repos
- Abusive automated scanning beyond quotas
- Webhook spoofing attempts
- Token theft from insecure client storage (mitigated by design)

7.4 Threats Affecting Major Assets

Table 7.1: Threats Affecting Major Assets

Asset	Possible Threat	Vulnerability	Potential Risk
Web Application Repo	Injection attacks	Unsanitized sinks (SQLi/XSS)	Data theft or account abuse
Dependency Lockfile	Known CVE exploit	Outdated vulnerable package	Remote compromise via library
Config / .env in git	Secret harvesting	Hardcoded API keys/tokens	Cloud/account takeover
Auth Module	Credential stuffing	Weak password policy / no MFA	Unauthorized platform access
GitHub Token Store	Token theft	Plaintext storage (anti-pattern)	Repo exfiltration
Webhook Endpoint	Event spoofing	Missing HMAC verification	Unauthorized scan triggers / DoS

7.5 Asset → Threat → Vulnerability → Risk Relationship

An asset (e.g., a web API repository) may be affected by a threat (e.g., injection or secret theft) that exploits a vulnerability (e.g., unsanitized query or hardcoded key). The resulting risk is scored and tracked until controls reduce residual open findings.

The relationship can be represented as:

Asset → Threat → Vulnerability → Risk Score → Control / Suppression → Residual Open Risk

8. VULNERABILITY ANALYSIS

8.1 Vulnerability Identification

Vulnerabilities are identified automatically by engines and optionally by AI review. Each finding is normalized with engine, rule, path/line, severity, family (e.g., sqli, xss, secret), and remediation hints.

8.2 Vulnerability Classification

Findings are classified by severity for prioritization within the risk register and dashboards:

- Critical – Severe compromise potential (e.g., production secrets, RCE-class SAST).
- High – Significant CIA impact (e.g., SQLi/XSS sinks, high CVEs).
- Medium – Moderate issues requiring planned remediation.
- Low / Info – Limited impact or hygiene findings still tracked.

8.3 Vulnerability Analysis Table

Table 8.1: Vulnerability Analysis

Affected Component	Vulnerability	Severity	Possible Impact	Proposed Countermeasure
Target Web App Code	SQL/XSS sinks (SAST)	High	Data exposure / session theft	Fix sinks; Semgrep gate in CI
Dependencies	Known CVEs (OSV)	High	Supply-chain compromise	Upgrade / replace packages
Git History / Config	Leaked secrets (Gitleaks)	Critical	Credential abuse	Rotate secrets; purge history
VERITAS Login	Brute force	Medium	Account takeover	Rate limits; Argon2; approval gate
Workspace API	IDOR on scans	High	Cross-user data access	Owner checks server-side
GitHub Connect	Foreign repo attach	High	Unauthorized code clone	Live ownership verification
Webhook Route	Unsigned push	High	Spurious jobs / load	X-Hub-Signature-256 HMAC

9. RISK ASSESSMENT

9.1 Risk Assessment Method

Risk assessment combines tool severity with project criticality. For each open finding, VERITAS computes a bounded score used for ordering and summaries:

risk_score = min(10, SEVERITY_BASE × CRITICALITY_WEIGHT)
Inherent scan risk ≈ aggregate of open finding scores / max_risk in summary

9.2 Risk Register

Table 9.1: Risk Register

Asset	Threat	Vulnerability	Likelihood	Impact	Risk	Control	Residual Risk
payments-api	Secret theft	Hardcoded key	5	5	9.5	Rotate + Gitleaks gate	3.0
payments-api	Injection	SQLi sink	4	5	8.0	Parameterized queries	3.0
portal-web	XSS	Unescaped render	4	4	8.0	Output encoding	3.0
auth-service	CVE exploit	Outdated JWT lib	3	5	8.0	Dependency upgrade	3.0
VERITAS API	IDOR	Missing owner check	3	5	8.0	Authz middleware	2.0
Webhook	Spoofing	No signature	3	4	5.5	HMAC verify	1.0

9.3 Security Controls

Security controls include fixing code, rotating secrets, upgrading dependencies, marking false positives, and suppressing accepted fingerprints. Platform controls (authn/z, encryption, quotas, webhook HMAC) protect the assessment system itself.

9.4 Residual Risk

Residual risk is observed as remaining open (non-fixed, non-false-positive) findings after remediation and re-scan. Compare APIs show added/removed fingerprints; metrics tables record before/after reductions.

10. IMPLEMENTATION

10.1 Hardware Requirements

- Processor: Intel Core i5 / Apple Silicon / equivalent or above (lab hosts; Linux preferred for Semgrep)
- RAM: Minimum 8 GB (16 GB recommended for concurrent scans)
- Storage: Minimum 15 GB free (clones + node modules + Python env)
- Display: Standard monitor for SPA dashboards
- Network: Internet access for GitHub OAuth, optional AI APIs, and CVE DB updates

10.2 Software Requirements

Software requirements cover OS, runtimes, frameworks, scanners, and tools used to build and validate VERITAS.

- Operating System: Windows (UI/dev), Linux/macOS recommended for full Semgrep engine
- Programming Languages: Python 3.11+, JavaScript/TypeScript (React)
- Frontend: Vite + React SPA (admin/user/auth)
- Backend: FastAPI + Uvicorn + SQLAlchemy
- Database: SQLite (lab default) / PostgreSQL (production)
- Code Editor: Visual Studio Code / Cursor
- Browser: Google Chrome, Microsoft Edge, or equivalent
- Version Control & Scanners: Git/GitHub; Gitleaks; OSV-Scanner; Semgrep; pytest; npm

10.3 Module Implementation

Authentication, admin directory, and profile modules establish identity. The workspace module implements projects and GitHub connect. Scan runner services orchestrate clone → engines → optional AI → persistence.

Findings APIs support status updates, suppressions, compare, cancel, and exports. Webhook routes enable auto-scan on push. The React SPA renders dashboards, progressive project creation, scan history, and finding drawers with GitHub blob deep links.

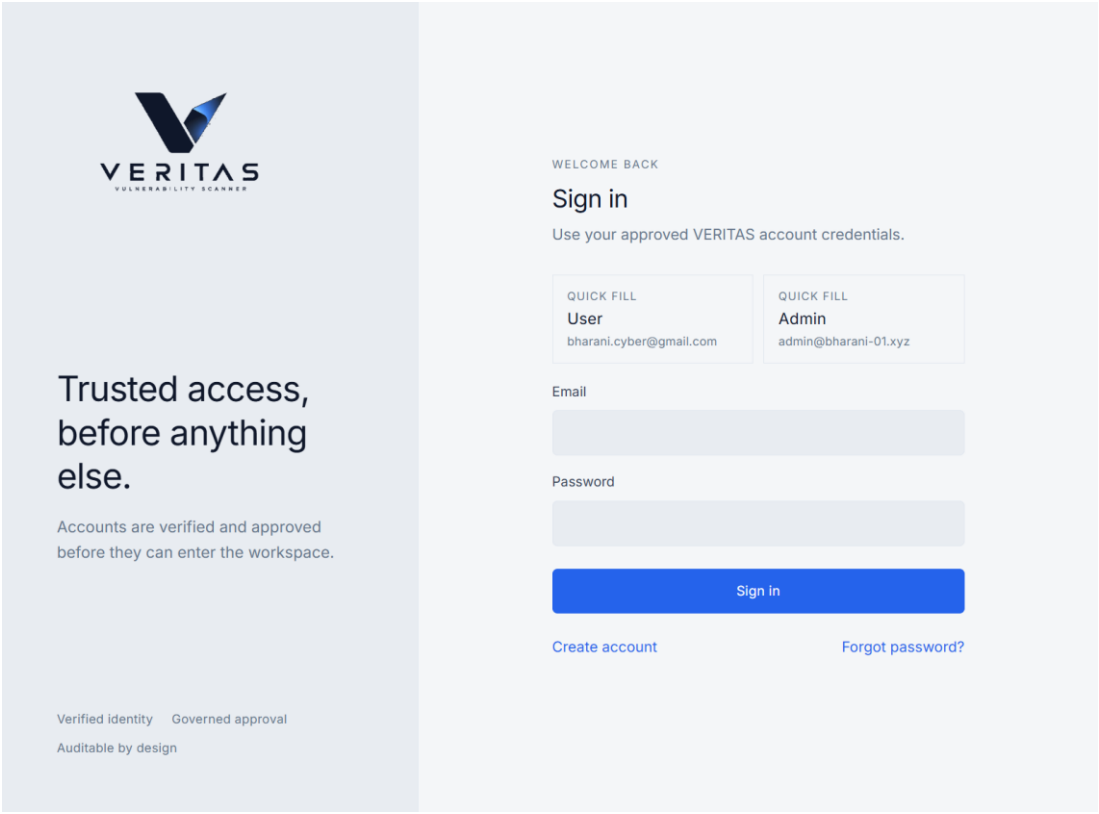


Fig 10.1: User Login Interface (live VERITAS)

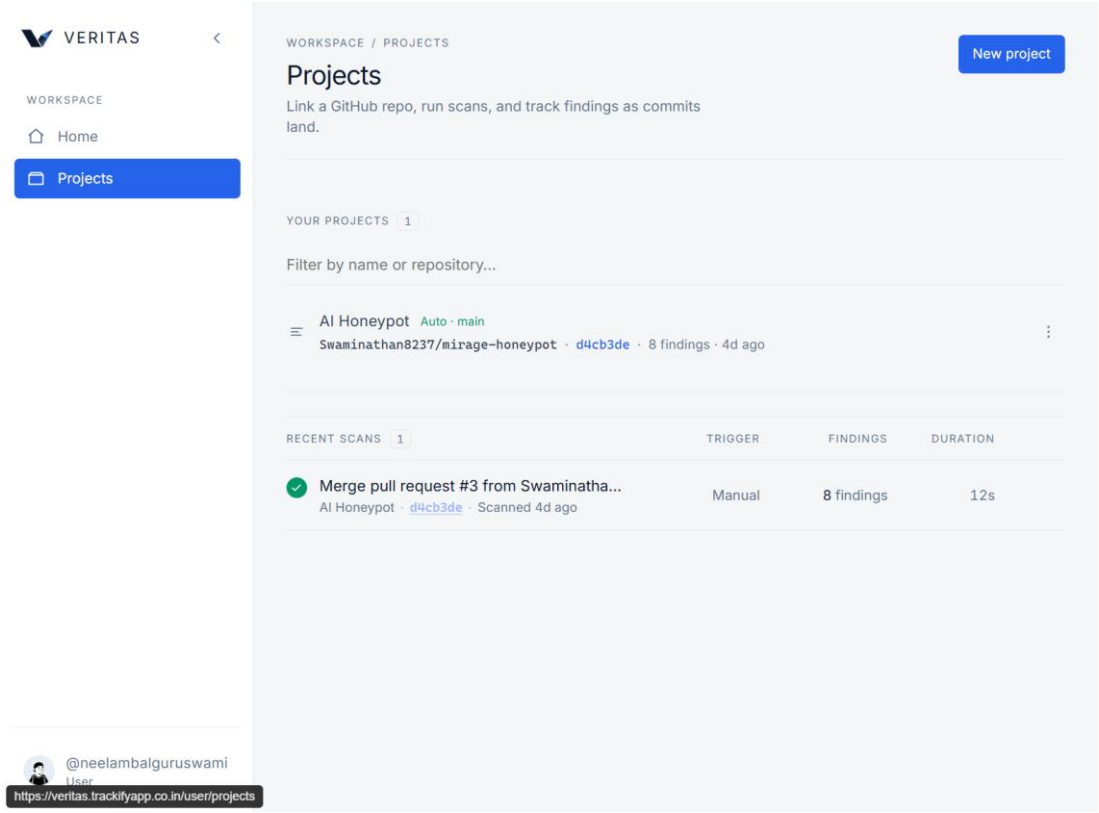


Fig 10.2: User Projects Workspace (live production VERITAS)

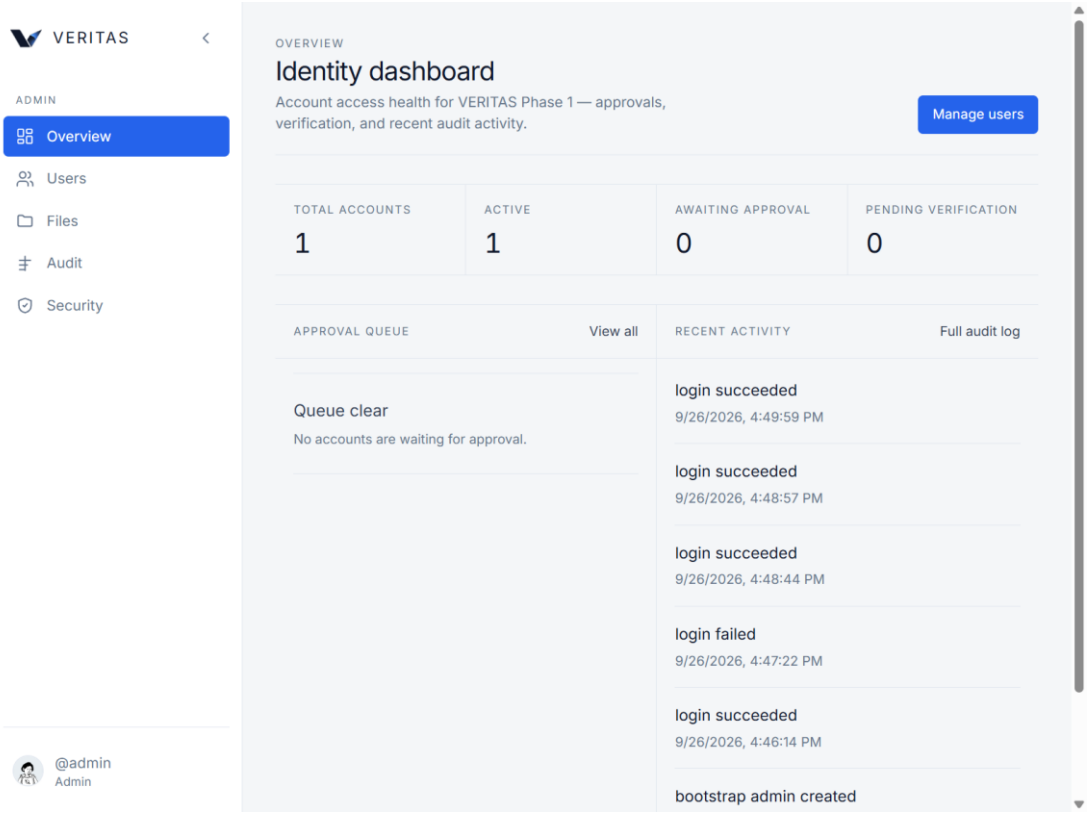


Fig 10.3: Administrator Dashboard (live VERITAS)

11. SECURITY IMPLEMENTATION

11.1 Security Mechanisms

VERITAS implements defense-in-depth for the platform that hosts vulnerability data:

- Authentication: Argon2 passwords; email verify; admin approval before first login.
- Authorization: role==admin for admin APIs; project ownership for workspace APIs.
- Access Control: separate admin/user SPAs; path-safe admin Files mounts.
- Input Validation: server-side schema/validation on all mutating endpoints.
- Secure Data Handling: hashed sessions/OTTs; encrypted GitHub tokens; no secrets in client.
- Session Management: HttpOnly veritas_session cookie; TTL; reset revokes all sessions.
- Logging: append-only audit_events; metadata-only http_request_events (no bodies/tokens).
- Monitoring: dashboards, Security telemetry graphs, notifications, scan quotas.
- Integrity: webhook HMAC; Security AI fail-open; SARIF/export without privileged creds.

These mechanisms reduce unauthorized access, forged repo attachment, token leakage, webhook spoofing, and abusive scan volume while preserving laboratory usability. In short, the scanner platform itself is treated as a security-sensitive application, not only as a tool that inspects other repositories.

11.2 Why Platform Security Matters in This Project

Because VERITAS stores repository metadata, findings, and GitHub connection state, a weakness in the platform could expose more than a single student repository. We therefore implement controls that are easy to explain in a laboratory report: verify the user, check ownership before attach, keep tokens server-side, and record important actions in audit logs.

In practice, the most important checks are (i) session cookies that students cannot read from JavaScript, (ii) admin routes that reject non-admin roles, and (iii) project APIs that reject foreign repository IDs. These checks were exercised during security testing and are listed again in Chapter 12.

11.3 Summary of Controls Used

Identity controls cover signup verification, admin approval, and Argon2 password hashing. Workspace controls cover owned-repo attach and encrypted GitHub tokens. Scan controls cover rate limits and soft-skip of missing engines so a lab machine without every binary still returns partial

results. Reporting controls cover capability-token report links and exports that do not embed privileged credentials. Together these layers support the residual-risk story used later in the results chapter.

The chapter therefore shows that security implementation is not only a list of tools, but a set of design decisions that keep the vulnerability-assessment workflow trustworthy for classroom use.

12. SECURITY TESTING

12.1 Testing Approach

Security testing was performed in an authorized laboratory setting against the VERITAS application, focusing on authz bypass attempts, session handling, input validation, webhook authenticity, and correctness of risk calculations.

The testing process follows:

Security Problem → Detection / Analysis → Countermeasure → Retesting → Result

12.2 Security Test Cases

Table 12.1: Security Test Cases

Test Case	Expected Result	Actual Result	Status	Evidence
Unauthenticated admin API	401/403 denied	Access denied	Pass	pytest + manual
Foreign github_repo_id attach	Reject after ownership check	Rejected	Pass	Workspace API log
Invalid login brute attempts	429 after limit	Rate limited	Pass	Audit rate_limited
Unsigned GitHub webhook	Request rejected	HMAC failure	Pass	Webhook tests
Risk score calculation	severity×criticality bounded ≤10	Scores matched	Pass	normalize.risk_score
Residual risk after fixes	Open count/max_risk decrease on re-scan	See NurseQuest metrics	Pass	Compare + Table 13.1
Session cookie flags	HttpOnly session cookie set	Flags present	Pass	Browser devtools

Test cases confirm that unauthorized access, forged repository IDs, invalid inputs, and unsigned webhooks are rejected, and that risk scoring / residual-risk views behave as designed.

13. RESULTS AND ANALYSIS

In this project, the implemented AI Based Web Application Vulnerability Detection and Risk Assessment platform (VERITAS) successfully supports the laboratory workflow: secure onboarding, GitHub-linked projects, multi-engine scans, optional AI reports, findings management, and dashboards. Evidence comes from a live production NurseQuest scan and from a live production AI Honeypot (mirage-honeypot) scan, with a supplementary local clone check.

Quantitative scan metrics are taken from the live NurseQuest production scan (35 findings, risk score 8) and from the live AI Honeypot scan on veritas.trackifyapp.co.in. Values are summarized in Table 13.1 and matching UI screenshots. We do not invent numerical reductions. Platform-oriented security checks are reported in the test table.

13.1 Results Table

Table 13.1: Results Table / Metrics

Metric	Observed value	Notes	Evidence
Live NurseQuest (production VERITAS)	Swaminathan8237/NurseQuest	Commit 69d678b; in-app GitHub scan on veritas.trackifyapp.co.in	report_figures/live_prod_nursequest_scan_metrics.json
Live NurseQuest findings / risk	35 findings; risk score 8.0	High=7, Medium=19, Info=9 (gitleaks + semgrep)	live_prod_scan_nursequest.png
Lab NurseQuest clone (supplementary)	detect-secrets=3, bandit=0	Read-only local clone (559 files); 3 engines skipped on Windows host	report_figures/nursequest_scan_metrics.json
Live AI Honeypot (production)	Swaminathan8237/mirage-honeypot	Git d4cb3de; 8 findings; risk 8 (H=6, M=2)	report_figures/live_prod_scan_metrics.json

On the live production deployment (veritas.trackifyapp.co.in) we scanned Swaminathan8237/NurseQuest (commit 69d678b) through the in-app GitHub workspace. The completed NurseQuest scan reported 35 findings (high=7, medium=19, info=9) with an overall risk score of 8.0. As a supplementary laboratory check, a read-only local clone (559 files) produced 3 detect-secrets hits and 0 Bandit findings on the Windows lab host. Separately, the live AI Honeypot scan of Swaminathan8237/mirage-honeypot (git d4cb3de) reported 8 findings (high=6, medium=2) with risk score 8. These values are recorded in Table 13.1 and were not invented for the report.

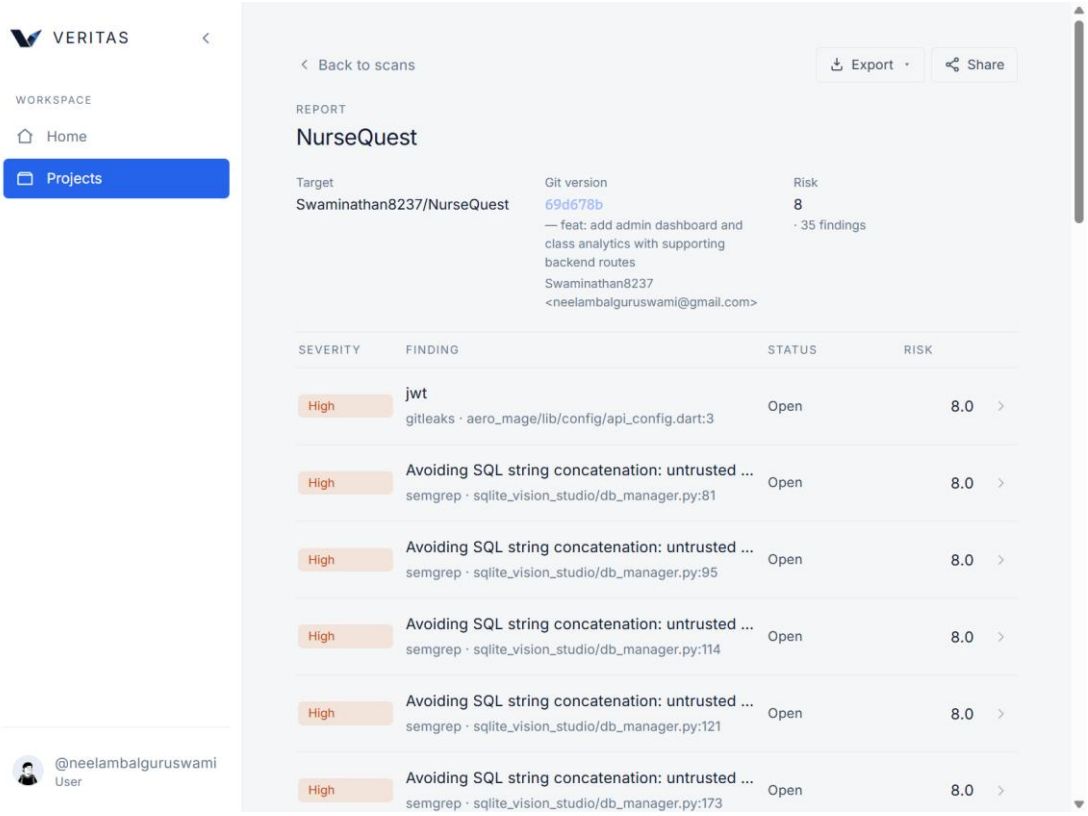


Fig 13.1: Live Production Scan Findings — NurseQuest (35 findings, risk 8)

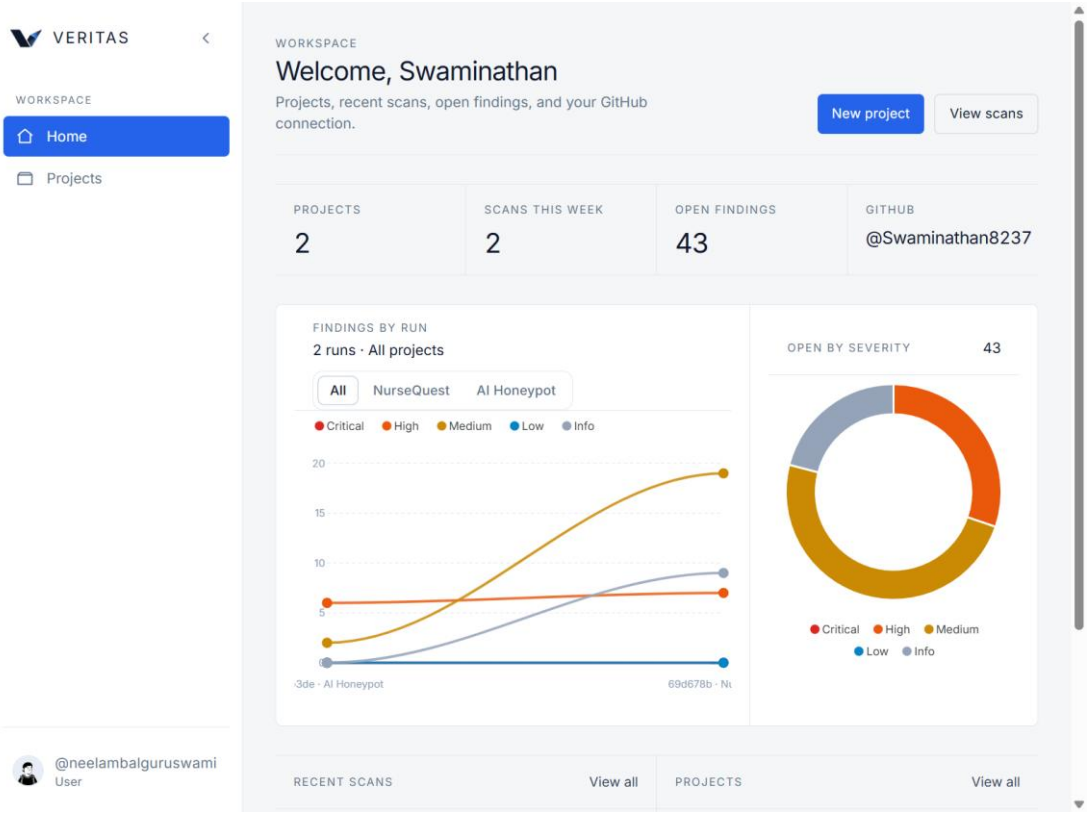


Fig 13.2: Live Production Home — Findings by Run and Severity

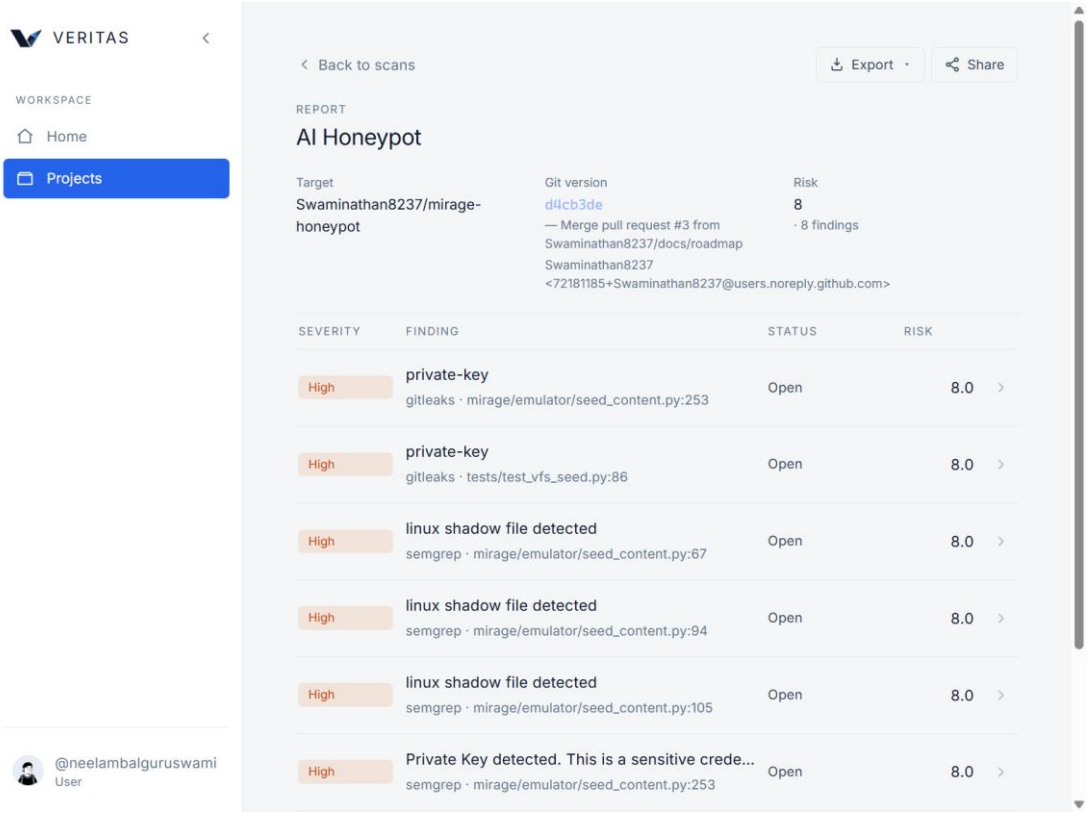


Fig 13.3: Live Production Scan Findings — AI Honeypot (8 findings, risk 8)

14. CONCLUSION

In this project, we attempt to design and implement AI Based Web Application Vulnerability Detection and Risk Assessment under the system name VERITAS. The work treats web application security assessment as a laboratory workflow rather than as an isolated scanner command. Users authenticate to a shared platform, connect owned GitHub repositories, run multi-engine detection, optionally request AI-assisted triage, and review findings with severity and risk context. In our view, this structure is closer to how a student team would practise vulnerability detection and residual-risk discussion during an Information Security laboratory course.

From the literature survey, earlier research such as static analysis with data mining (Medeiros et al.), DEKANT, TAP, VulEye, and RealVul mainly improves detection quality for specific languages or model families. Those papers are valuable, but they do not fully solve the student-lab need for identity, authorization, repository ownership checks, finding lifecycle, and evidence that can be pasted into a report. Our contribution is therefore more systems-oriented: we try to connect trusted engines, optional AI reporting, and residual-risk tracking inside one authenticated workspace.

During implementation we also learned practical lessons about laboratory constraints. Some engines may be missing on a Windows host, OAuth tokens can expire, and live production evidence can differ from a sparse local clone. For this reason the report presents both the live NurseQuest scan on veritas.trackifyapp.co.in (35 findings, risk score 8) and the live AI Honeypot scan (8 findings, risk score 8), together with a supplementary local clone check. We prefer this mixed evidence over inventing neat before/after percentages.

Security testing of the platform itself focused on unauthorized access, ownership checks, session handling, webhook authenticity, and risk-score behaviour. The results support the claim that basic platform controls work in an authorized lab setting, while also showing that residual open findings remain visible until remediation or justified suppression. This residual-risk view is important for student reports because it prevents the false impression that one scan automatically means a clean application.

Overall, we conclude that VERITAS meets the academic objective of an end-to-end vulnerability detection and risk assessment system suitable for Information Security laboratory study. Detection quality still depends on installed engines and model configuration, and AI assistance must remain optional and fail-open. Even with those limits, the platform gives a clear path from repository attachment to findings, risk discussion, and report evidence. Future work can deepen DAST coverage and portfolio management without discarding the current identity, scan, and residual-risk foundation.

15. FUTURE ENHANCEMENT

Based on the limitations observed during implementation and live testing, possible future enhancements for VERITAS are listed below. Each point is written as a short description of what can be added and why it would help the laboratory platform.

- Deeper DAST / authenticated HTTP crawling modules: Beyond repository static analysis, a careful authenticated crawler could exercise login-protected routes and capture runtime findings that source scanners may miss, while still remaining inside an authorized lab scope.
- Organization GitHub Apps and multi-repo portfolios: Supporting GitHub Apps and portfolio views would help laboratories manage many student repositories under one organization without repeating OAuth connect steps for every project.
- Stronger ASPM-style trending and SLA tracking: Time-series charts for open risk, mean time to fix, and simple SLA timers would make residual-risk discussion easier across weekly lab submissions.
- SIEM integration for platform audit streaming: Forwarding audit events to a SIEM would let administrators correlate VERITAS activity with broader campus security monitoring.
- Expanded AI agents with retrieval over secure playbooks: Grounding AI suggestions in approved remediation playbooks could reduce hallucinated advice and keep student guidance consistent with course policy.
- Container/IaC scanning engines alongside SAST/SCA: Adding Dockerfile, Kubernetes, and Terraform checks would cover deployment configuration risks that often accompany modern web applications.
- High-availability workers and shared rate-limit stores: Separating scan workers and using a shared rate-limit store would support concurrent class usage without uneven scan queues.
- Mobile-responsive analyst apps and accessibility upgrades: Improving responsive layouts and accessibility would make findings review usable on tablets and for students who rely on assistive technologies.
- Automated ticket creation in issue trackers: Opening GitHub Issues or lab tickets from a finding would connect detection directly to remediation homework and peer review.

- Broader multilingual report templates: Export templates in additional languages would help mixed-language cohorts prepare submission packages without rewriting captions manually.

16. REFERENCES

1. I. Medeiros, N. Neves, and M. Correia, "Detecting and Removing Web Application Vulnerabilities with Static Analysis and Data Mining," IEEE Transactions on Reliability, vol. 65, no. 1, pp. 54–69, 2016.
2. I. Medeiros, N. Neves, and M. Correia, "DEKANT: A Static Analysis Tool that Learns to Detect Web Application Vulnerabilities," in Proc. 25th Int. Symp. Software Testing and Analysis (ISSTA), ACM, 2016.
3. Y. Fang, S. Han, C. Huang, and R. Wu, "TAP: A static analysis model for PHP vulnerabilities based on token and deep learning technology," PLOS ONE, vol. 14, no. 11, e0225196, 2019.
4. C. Lin, Y. Xu, Y. Fang, and Z. Liu, "VulEye: A Novel Graph Neural Network Vulnerability Detection Approach for PHP Application," Applied Sciences, vol. 13, no. 2, art. 825, 2023.
5. D. Cao, Y. Liao, and X. Shang, "RealVul: Can We Detect Vulnerabilities in Web Applications with LLM?," arXiv preprint arXiv:2410.07573, 2024.
6. N. Jovanovic, C. Kruegel, and E. Kirda, "Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities," in Proc. IEEE Symp. Security and Privacy (S&P), 2006.
7. V. B. Livshits and M. S. Lam, "Finding Security Vulnerabilities in Java Applications with Static Analysis," in Proc. USENIX Security Symposium, 2005.
8. Y. Zheng and X. Zhang, "Path Sensitive Static Analysis of Web Applications for Remote Code Execution Vulnerability Detection," in Proc. Int. Conf. Software Engineering (ICSE), 2013.
9. L. K. Shar, H. B. K. Tan, and L. C. Briand, "Mining SQL Injection and Cross Site Scripting Vulnerabilities using Hybrid Program Analysis," in Proc. Int. Conf. Software Engineering (ICSE), 2013.
10. G. Grieco, G. L. Grinblat, L. Uzal, S. Rawat, J. Feist, and L. Mounier, "Toward Large-Scale Vulnerability Discovery using Machine Learning," in Proc. ACM CODASPY, 2016.
11. Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A Deep Learning-Based System for Vulnerability Detection," in Proc. NDSS, 2018.

12. R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated Vulnerability Detection in Source Code Using Deep Representation Learning," in Proc. IEEE ICMLA, 2018.
13. S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep Learning Based Vulnerability Detection: Are We There Yet?," IEEE Transactions on Software Engineering, vol. 48, no. 9, pp. 3280–3296, 2022.
14. H. Hanif, M. H. N. M. Nasir, M. F. Ab Razak, A. Firdaus, and N. B. Anuar, "The Rise of Software Vulnerability: Taxonomy of Software Vulnerabilities Detection and Machine Learning Approaches," Journal of Network and Computer Applications, vol. 179, 2021.
15. Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," in Proc. NeurIPS, 2019.